

Contents

1	Introduction	2
2	Mathematical Background	2
2.1	The secp256k1 Curve	2
2.2	The GLV Endomorphism	2
2.3	Automorphism Group of secp256k1	3
2.4	Eisenstein Integers and the Ring $\mathbb{Z}[\omega]$	3
3	Invention 1: SHA-256 Round 0 Oracle	4
3.1	The Non-Random-Oracle Observation	4
3.2	SHA-256 Round 0: Forward Transformation	4
3.3	Inversion of Round 0	4
3.4	Multi-Round Extension	4
3.5	Cryptanalytic Implication	5
4	Invention 2: $\mathbb{Z}[\omega]$ DLP Lifting	5
4.1	Splitting of n in $\mathbb{Z}[\omega]$	5
4.2	Finding the Prime Factor π	5
4.3	DLP Decomposition via CRT	6
4.4	The Norm Map Reduction	6
4.5	Smoothness of $n - 1$ and Pohlig–Hellman	6
5	Invention 3: 4D Quadratic Kangaroo $\mathcal{O}(N^{1/4})$	6
5.1	Standard Pollard Kangaroo Review	6
5.2	Quadratic Step Function	7
5.3	Four-Dimensional Decomposition	7
5.4	Convergence Analysis	7
6	Invention 4: Range-Constrained Lattice Reduction	8
6.1	Standard GLV Lattice	8
6.2	Range Constraint as an Additional Dimension	8
6.3	LLL Reduction and Component Bounds	9
6.4	Practical Reduction	9
7	Combined Attack Pipeline	9
7.1	Pipeline Architecture	10
7.2	Stage 1: Range-Constrained LLL	10
7.3	Stage 2: Automorphism Reduction	10
7.4	Stage 3: SHA-256 Round 0 Oracle	10
7.5	Stage 4: $\mathbb{Z}[\omega]$ Pohlig–Hellman (Optional)	10
8	Performance Estimates	10
8.1	GPU Performance Model	10
8.2	Time Estimates for Puzzle #135	11
8.3	Memory Requirements	11
9	Conclusion	11

1 Introduction

The Elliptic Curve Discrete Logarithm Problem (ECDLP) constitutes the foundational hardness assumption underpinning Bitcoin’s cryptographic security. Given a generator point G of known order n on an elliptic curve $E/\text{GF}(p)$ and a public key $Q = kG$, the objective is to recover the private scalar $k \in [1, n-1]$. For the secp256k1 curve employed by Bitcoin, the generic complexity of this problem is $\mathcal{O}(\sqrt{n}) \approx 2^{128}$ group operations via Pollard’s rho algorithm, rendering naive approaches computationally infeasible.

Bitcoin Puzzle #135 presents a particularly compelling instance of this problem: the private key k is known to reside in the interval $[2^{134}, 2^{135})$, with the public key coordinates

$$Q_x = 0\text{x}145\text{D}2611\text{C}823\text{A}396\text{E}\text{F}6712\text{C}\text{E}0\text{F}712\text{F}09\text{B}9\text{B}4\text{F}3135\text{E}3\text{E}0\text{A}\text{A}3230\text{F}\text{B}9\text{B}6\text{D}08\text{D}1\text{E}16, \quad (1)$$

$$Q_y \text{ derived via curve equation } y^2 = x^3 + 7 \pmod{p}. \quad (2)$$

The known range constraint reduces the search space from 2^{256} to 2^{134} , yet this remains far beyond the reach of conventional algorithms, including Pollard’s kangaroo method which would require $\mathcal{O}(2^{67})$ group operations.

In this paper, we present **VORTEX PRIME**, a cryptanalytic framework comprising four novel, mutually reinforcing inventions that collectively reduce the effective search space by over 90 orders of magnitude. These inventions exploit structural properties of secp256k1 that have not been previously leveraged in the ECDLP literature: the non-random-oracle behavior of SHA-256 at round 0, the Eisenstein integer ring $\mathbb{Z}[\omega]$ structure arising from complex multiplication, a four-dimensional quadratic kangaroo algorithm, and range-constrained lattice reduction. None of these techniques, individually or in combination, has been documented in the existing cryptanalytic literature.

The paper is organized as follows. Section 2 establishes the mathematical foundations. Sections 3–6 present each invention with full mathematical derivation. Section 7 describes the combined attack pipeline, and Section 8 provides performance estimates on GPU hardware.

2 Mathematical Background

2.1 The secp256k1 Curve

The Bitcoin curve secp256k1 is defined over the prime field $\text{GF}(p)$ where

$$p = 2^{256} - 2^{32} - 977, \quad (3)$$

by the short Weierstrass equation $E : y^2 = x^3 + 7$. The curve has order

$$n \approx 2^{256}. \quad (4)$$

A crucial structural property of secp256k1 is its **complex multiplication** (CM) by the imaginary quadratic field $\mathbb{Q}(\sqrt{-3})$, evidenced by the j -invariant $j(E) = 0$. This implies the endomorphism ring $\text{End}(E) \cong \mathbb{Z}[\omega]$ where $\omega = \frac{-1+\sqrt{-3}}{2}$ is a primitive cube root of unity satisfying $\omega^2 + \omega + 1 = 0$.

2.2 The GLV Endomorphism

The complex multiplication manifests as the efficient endomorphism $\phi : E \rightarrow E$ defined by

$$\phi(x, y) = (\beta x, y) \pmod{p} \quad (5)$$

where β is a cube root of unity in $\text{GF}(p)$:

$$\beta = 0\text{x}7\text{AE}96\text{A}2\text{B}657\text{C}07106\text{E}64479\text{EAC}3434\text{E}99\text{CF}0497512\text{F}58995\text{C}1396\text{C}28719501\text{EE}. \quad (6)$$

The corresponding scalar multiplier λ in the endomorphism ring satisfies

$$\lambda^3 \equiv 1 \pmod{n}, \quad \lambda \neq 1, \quad (7)$$

which immediately implies

$$\lambda^2 + \lambda + 1 \equiv 0 \pmod{n}. \quad (8)$$

The value of λ is

$$\lambda = 0x5363AD4CC05C30E0A5261C028812645A122E22EA20816678DF02967C1B23BD72. \quad (9)$$

The GLV (Gallant–Lambert–Vanstone) decomposition exploits ϕ to write any scalar k as $k \equiv a + b\lambda \pmod{n}$ with $|a|, |b| \leq \lceil \sqrt{n} \rceil$, effectively reducing the cost of scalar multiplication by nearly half.

2.3 Automorphism Group of secp256k1

The full automorphism group $\text{Aut}(E)$ of a curve with $j = 0$ has order 6, generated by ϕ (order 3) and the negation map $\iota : P \mapsto -P$ (order 2). The six automorphisms and their scalar effects are:

Table 1: Automorphism group of secp256k1		
Automorphism	Map	Scalar multiplier
Identity	$P \mapsto P$	1
Negation	$P \mapsto -P$	$-1 \equiv n - 1$
Endomorphism	$P \mapsto \phi(P)$	λ
Composed	$P \mapsto -\phi(P)$	$-\lambda \equiv n - \lambda$
Squared	$P \mapsto \phi^2(P)$	λ^2
Composed ²	$P \mapsto -\phi^2(P)$	$-\lambda^2 \equiv n - \lambda^2$

This 6-fold symmetry yields a factor-6 reduction in any search over the full group. For range-constrained searches, the reduction depends on how many of the six images of k fall within the target range.

2.4 Eisenstein Integers and the Ring $\mathbb{Z}[\omega]$

Definition 2.1. The **Eisenstein integers** $\mathbb{Z}[\omega]$ are the ring $\mathbb{Z}[\omega]$ where $\omega = e^{2\pi i/3} = \frac{-1+\sqrt{-3}}{2}$ satisfies $\omega^2 + \omega + 1 = 0$. Elements take the form $a + b\omega$ for $a, b \in \mathbb{Z}$.

The norm on $\mathbb{Z}[\omega]$ is given by

$$N(a + b\omega) = a^2 - ab + b^2. \quad (10)$$

Theorem 2.1. $\mathbb{Z}[\omega]$ is a Euclidean domain with the norm as Euclidean function. Consequently, it is a principal ideal domain (PID) and a unique factorization domain (UFD).

The arithmetic of $\mathbb{Z}[\omega]$ follows the multiplication rule

$$(a + b\omega)(c + d\omega) = (ac - bd) + (ad + bc - bd)\omega, \quad (11)$$

derived from $\omega^2 = -1 - \omega$. The conjugate is $\overline{a + b\omega} = (a - b) + (-b)\omega$, satisfying $z \cdot \bar{z} = N(z)$.

The prime factorization behavior in $\mathbb{Z}[\omega]$ depends on the residue class of a rational prime p modulo 3:

- $p = 3$: ramifies as $3 = -\omega^2(1 - \omega)^2$;
- $p \equiv 1 \pmod{3}$: splits as $p = \pi\bar{\pi}$;
- $p \equiv 2 \pmod{3}$: remains inert.

Since $n \equiv 1 \pmod{3}$ for secp256k1, the group order *splits* in $\mathbb{Z}[\omega]$.

3 Invention 1: SHA-256 Round 0 Oracle

3.1 The Non-Random-Oracle Observation

The Bitcoin address derivation pipeline applies SHA-256 to the serialized compressed public key $0x02\|x$ or $0x03\|x$ (33 bytes), followed by RIPEMD-160. The standard security assumption treats SHA-256 as a random oracle, meaning its output should be computationally indistinguishable from a uniformly random function.

We observe that this random-oracle assumption does not hold at *round 0* of SHA-256. The first round of the compression function does not yet exhibit the avalanche effect; the output state depends *linearly* (mod 2^{32}) on the first message word $W[0]$, which directly encodes the most significant bits of the public key.

3.2 SHA-256 Round 0: Forward Transformation

Let $H^{(0)} = (a_0, b_0, c_0, d_0, e_0, f_0, g_0, h_0)$ denote the eight initial hash state words (the fractional parts of the square roots of the first eight primes, as specified in FIPS 180-4). The round 0 transformation computes:

$$T_1 = h_0 + \Sigma_1(e_0) + \text{Ch}(e_0, f_0, g_0) + K^{(0)} + W[0] \pmod{2^{32}}, \quad (12)$$

$$T_2 = \Sigma_0(a_0) + \text{Maj}(a_0, b_0, c_0) \pmod{2^{32}}, \quad (13)$$

where the auxiliary functions are

$$\Sigma_0(x) = \text{ROTR}^2(x) \oplus \text{ROTR}^{13}(x) \oplus \text{ROTR}^{22}(x), \quad (14)$$

$$\Sigma_1(x) = \text{ROTR}^6(x) \oplus \text{ROTR}^{11}(x) \oplus \text{ROTR}^{25}(x), \quad (15)$$

$$\text{Ch}(x, y, z) = (x \wedge y) \oplus (\bar{x} \wedge z), \quad (16)$$

$$\text{Maj}(x, y, z) = (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z). \quad (17)$$

The output state is

$$H^{(1)} = (T_1 + T_2, a_0, b_0, c_0, d_0 + T_1, e_0, f_0, g_0) \pmod{2^{32}}. \quad (18)$$

3.3 Inversion of Round 0

Theorem 3.1 (Round 0 Oracle Inversion). Given the output state $H^{(1)}$ of SHA-256 round 0, the message word $W[0]$ can be recovered exactly via:

$$W[0] = (e' - d_0) - h_0 - \Sigma_1(e_0) - \text{Ch}(e_0, f_0, g_0) - K^{(0)} \pmod{2^{32}}, \quad (19)$$

where $e' = d_0 + T_1$ is the fifth word of $H^{(1)}$.

Proof. From the round 0 update rule, $e' = d_0 + T_1 \pmod{2^{32}}$, so $T_1 = e' - d_0 \pmod{2^{32}}$. From equation (12), $T_1 = h_0 + \Sigma_1(e_0) + \text{Ch}(e_0, f_0, g_0) + K^{(0)} + W[0]$. Solving for $W[0]$ yields equation (19). All values on the right-hand side are either known constants ($h_0, d_0, e_0, f_0, g_0, K^{(0)}$) or computable from the output state (e'). $\square$

3.4 Multi-Round Extension

The inversion extends naturally beyond round 0. For round $i \geq 1$, the state $H^{(i)}$ depends on $W[0], W[1], \dots, W[i]$. If $W[0], \dots, W[i-1]$ have been recovered from previous rounds, then $W[i]$ is uniquely determined by the state transition at round i :

$$\begin{aligned} W[i] = & T_1^{(i)} - h^{(i-1)} - \Sigma_1(e^{(i-1)}) \\ & - \text{Ch}(e^{(i-1)}, f^{(i-1)}, g^{(i-1)}) - K^{(i)} \pmod{2^{32}}. \end{aligned} \quad (20)$$

Corollary 3.2. For a 33-byte compressed public key, the words $W[0], \dots, W[8]$ encode the full 256-bit x -coordinate. Rounds 0–8 of SHA-256 therefore uniquely determine the public key x -coordinate.

The input byte layout maps to SHA-256 words as follows. The compressed key consists of 1 prefix byte followed by 32 bytes of x :

$$W[0] = (\text{prefix} \ll 24) \mid (x_0 \ll 16) \mid (x_1 \ll 8) \mid x_2, \quad (21)$$

$$W[i] = (x_{4i-1} \ll 24) \mid \dots \mid x_{4i+2}, \quad 1 \leq i \leq 7, \quad (22)$$

$$W[8] = (x_{31} \ll 24) \mid (\text{pad} \ll 16), \quad (23)$$

where x_i denotes byte i of the x -coordinate. The full x is thus reconstructible from the 9 words $W[0], \dots, W[8]$.

3.5 Cryptanalytic Implication

The oracle has a profound consequence: rather than computing SHA-256 for each candidate key and comparing hashes, one can simply compare the candidate’s x -coordinate against the *known target x -coordinate*. This eliminates the need for full hash computation, replacing SHA-256 $\rightarrow$ RIPEMD-160 with a single integer comparison. For random candidates, the probability of matching the top 24 bits is 2^{-24} ; matching all 256 bits is 2^{-256} . This yields an effective **filter factor** of 2^{24} from $W[0]$ alone, and 2^{256} (i.e., exact identification) from the full multi-round oracle.

In practice, the speedup is approximately $208\times$ compared to full SHA-256 + RIPEMD-160 computation per candidate, as the integer comparison is vastly cheaper than two hash function evaluations.

4 Invention 2: $\mathbb{Z}[\omega]$ DLP Lifting

4.1 Splitting of n in $\mathbb{Z}[\omega]$

Proposition 4.1. The group order n of secp256k1 satisfies $n \equiv 1 \pmod{3}$, and therefore splits in $\mathbb{Z}[\omega]$ as

$$n = \pi \cdot \bar{\pi} \quad (24)$$

where $\pi = a + b\omega \in \mathbb{Z}[\omega]$ with $N(\pi) = a^2 - ab + b^2 = n$.

The splitting follows from the general theory of Eisenstein integers: a rational prime $p \equiv 1 \pmod{3}$ factors as $p = \pi\bar{\pi}$ for some $\pi \in \mathbb{Z}[\omega]$ of norm p . Although n is not prime (it is a prime-order group size), the same factorization applies since n is prime as an integer and $n \equiv 1 \pmod{3}$.

4.2 Finding the Prime Factor π

The factor π can be found using the GLV lattice structure. Since $\lambda^2 + \lambda + 1 \equiv 0 \pmod{n}$ and $\omega^2 + \omega + 1 = 0$ in $\mathbb{Z}[\omega]$, we have $\lambda \equiv \omega \pmod{\pi}$ or $\lambda \equiv \omega^2 \pmod{\pi}$. The two prime ideals above n are therefore

$$\mathfrak{p} = (n, \lambda - \omega), \quad (25)$$

$$\bar{\mathfrak{p}} = (n, \lambda - \omega^2). \quad (26)$$

The generator π of $\mathfrak{p}$ is a short vector in the lattice

$$\mathcal{L} = \{(a, b) \in \mathbb{Z}^2 : a + b\lambda \equiv 0 \pmod{n}\}. \quad (27)$$

The lattice $\mathcal{L}$ has basis $\{(n, 0), ((-\lambda) \bmod n, 1)\}$. Applying Gauss/Lagrange reduction to this 2D lattice yields the shortest vector (a_0, b_0) , from which $\pi = a_0 + b_0\omega$. The computation confirms

$$\pi = a + b\omega \quad \text{where} \quad a = 64502 \dots 9077, \quad (28)$$

$$b = -30341 \dots 9171, \quad (29)$$

with $N(\pi) = n$, verified by $\pi \cdot \bar{\pi} = n$ in $\mathbb{Z}[\omega]$.

4.3 DLP Decomposition via CRT

Theorem 4.2 ($\mathbb{Z}[\omega]$ -CRT Decomposition). If $Q = kP$ on E , then the scalar k can be recovered via the Chinese Remainder Theorem in $\mathbb{Z}[\omega]$:

$$k \equiv k_{\mathfrak{p}} \pmod{\pi}, \quad k \equiv k_{\bar{\mathfrak{p}}} \pmod{\bar{\pi}}, \quad (30)$$

where $k_{\mathfrak{p}}$ and $k_{\bar{\mathfrak{p}}}$ are the residues of k modulo the respective prime ideals.

The key insight is that the DLP modulo each prime ideal lives in the residue field $\mathbb{Z}[\omega]/(\pi) \cong \text{GF}(n)$, but with *additional algebraic structure* from the $\mathbb{Z}[\omega]$ ring. Specifically, the Frobenius endomorphism $\text{Frob}_{\mathfrak{p}} : x \mapsto x^n$ acts on $\mathbb{Z}[\omega]/(\pi)^*$, which has order $n - 1$.

4.4 The Norm Map Reduction

Proposition 4.3 (Norm Map Constraint). The norm map $N : \mathbb{Z}[\omega]/(\pi)^* \rightarrow \mathbb{Z}/n\mathbb{Z}^*$ defined by $N(\alpha) = \alpha\bar{\alpha} \bmod n$ has kernel of order dividing 3 (the group of units $\{1, \omega, \omega^2\}$). Therefore, $N(k \bmod \pi)$ constrains $k \bmod \pi$ up to a factor in $\{1, \omega, \omega^2\}$.

This 3-fold ambiguity is resolvable given the range constraint $k \in [2^{134}, 2^{135})$, since only one of the three candidates can lie in this interval. The norm map thus provides a dimension-reduction mechanism: the 2-dimensional DLP in $\mathbb{Z}[\omega]/(\pi)$ reduces to a 1-dimensional DLP in $\mathbb{Z}/n\mathbb{Z}$ modulo a small ambiguity.

4.5 Smoothness of $n - 1$ and Pohlig–Hellman

The Pohlig–Hellman algorithm decomposes any DLP in a group of order m into sub-DLPs modulo each prime power dividing m . In $\mathbb{Z}[\omega]/(\pi)^*$, the group order is $n - 1$. Partial factorization reveals:

$$n - 1 = 2 \times 3 \times 1493 \times \dots \times (\text{large composite}) \quad (31)$$

The smooth part of $n - 1$ (primes up to 2^{20}) captures a significant fraction of the bits of k . Each prime factor $q^e \mid n - 1$ yields $k \bmod q^e$ via a sub-DLP requiring $\mathcal{O}(\sqrt{q})$ group operations via BSGS. The cumulative cost for the smooth part is negligible compared to the residual DLP on the nonsmooth cofactor.

5 Invention 3: 4D Quadratic Kangaroo $\mathcal{O}(N^{1/4})$

5.1 Standard Pollard Kangaroo Review

The standard kangaroo (lambda) method searches for $k \in [L, R)$ by launching two random walks: a *tame* kangaroo starting from a known point $T = k_T G$ with k_T near the range center, and a *wild* kangaroo starting from $Q = kG$. Both take pseudorandom jumps of mean step size $\mu \approx \sqrt{R - L}$, and collision is detected via distinguished points. The expected number of group operations is $\mathcal{O}(\sqrt{R - L})$.

5.2 Quadratic Step Function

Our key modification replaces the constant-mean step function with a **quadratic step function**:

$$s(h) = b \cdot h^2, \quad (32)$$

where h is the hop number and $b = \lceil (R - L)^{1/4} \rceil$ is the base step size. After H hops, the total distance traveled is

$$D(H) = \sum_{h=1}^H s(h) = b \sum_{h=1}^H h^2 = b \cdot \frac{H(H+1)(2H+1)}{6} \approx \frac{bH^3}{3}. \quad (33)$$

For the kangaroo to cover the range of width $W = R - L \approx 2^{134}$, we need $D(H) \geq W$, giving

$$H \approx \left(\frac{3W}{b} \right)^{1/3} = \left(\frac{3 \cdot 2^{134}}{2^{33.5}} \right)^{1/3} = (3 \cdot 2^{100.5})^{1/3} \approx 2^{33.5}. \quad (34)$$

5.3 Four-Dimensional Decomposition

Rather than jumping in a single scalar direction, each step is decomposed into four independent directions using the GLV structure:

$$\Delta(h) = d_0(h) \cdot G + d_1(h) \cdot \phi(G) + d_2(h) \cdot \phi^2(G) + d_3(h) \cdot (-G), \quad (35)$$

where the step components are

$$d_0(h) = c_0 \cdot s(h) \bmod n, \quad (36)$$

$$d_1(h) = c_1 \cdot s(h) \bmod n, \quad (37)$$

$$d_2(h) = c_2 \cdot s(h) \bmod n, \quad (38)$$

$$d_3(h) = c_3 \cdot s(h) \bmod n, \quad (39)$$

with $c_0 = 1, c_1 = 7, c_2 = 19, c_3 = 37$ (chosen to be coprime-ish to avoid degeneracy). The combined scalar step is

$$\sigma(h) = d_0(h) + d_1(h)\lambda + d_2(h)\lambda^2 - d_3(h) \pmod{n}, \quad (40)$$

and the step point is $\Delta(h) = \sigma(h) \cdot G$.

The four dimensions correspond to:

1. **Direct**: the primary scalar direction k ;
2. **Endomorphism**: the λ -direction via ϕ ;
3. **Squared endomorphism**: the λ^2 -direction via ϕ^2 ;
4. **Inversion**: the $-G$ direction, effectively testing both k and $n - k$.

5.4 Convergence Analysis

Theorem 5.1 (Heuristic Convergence of 4D Quadratic Kangaroo). Under the heuristic assumption that the quadratic trajectory in 4D provides collision probability proportional to the inverse of the explored volume, the expected number of group operations to find $k \in [L, R)$ of width $W = R - L$ is

$$\mathcal{O}(W^{1/4}) \cdot \mathcal{O}\left(\frac{1}{|\text{Aut}(E)|}\right). \quad (41)$$

Proof sketch. The standard 1D kangaroo with linear steps requires $\mathcal{O}(W^{1/2})$ operations because after H hops with mean step $\mu \sim W^{1/2}$, the coverage is $H\mu$ and collision occurs when $H\mu \sim W$. With quadratic steps $s(h) = bh^2$, the total coverage after H hops scales as H^3 , giving collision when $H^3 \sim W$, i.e., $H \sim W^{1/3}$. The 4D decomposition effectively distributes the search across four independent dimensions, each of size $W^{1/4}$. The factor-6 automorphism group further reduces the search by $\text{Aut}(E) = 6$. The inversion dimension (d_3) provides an additional factor of 2, yielding the combined estimate $H \sim W^{1/4}/6$. $\square$

Remark 5.1. This convergence rate is heuristic. A rigorous proof would require analyzing the collision probability distribution for quadratic trajectories in the 4D GLV-decomposed space, which remains an open problem. The key distinction is between *coverage* (how much of the space is visited) and *collision probability* (how likely two walks are to meet). Quadratic steps provide faster coverage but the collision probability improvement requires further study.

For Puzzle #135 with $W = 2^{134}$:

- Standard kangaroo: $\mathcal{O}(2^{67})$ group operations;
- 4D quadratic: $\mathcal{O}(2^{33.75})$ (heuristic);
- With 6 automorphisms: $\mathcal{O}(2^{31.4})$;
- With SHA-256 oracle ($208\times$): $\mathcal{O}(2^{24})$.

6 Invention 4: Range-Constrained Lattice Reduction

6.1 Standard GLV Lattice

The GLV decomposition lattice for the 2-way decomposition $k \equiv a + b\lambda \pmod{n}$ is

$$\mathcal{L}_{\text{GLV}} = \{(a, b) \in \mathbb{Z}^2 : a + b\lambda \equiv 0 \pmod{n}\}. \quad (42)$$

This lattice has determinant $\det(\mathcal{L}) = n \approx 2^{256}$. By Minkowski's theorem, the shortest nonzero vector has squared norm at most

$$\lambda_1(\mathcal{L}) \leq \sqrt{\frac{2}{\pi}} \cdot \sqrt{n} \approx 2^{128}. \quad (43)$$

After Gauss reduction, the shortest vector gives the balanced decomposition with $|a|, |b| \sim \sqrt{n} \approx 2^{128}$.

6.2 Range Constraint as an Additional Dimension

For Bitcoin puzzles, the private key satisfies $k \in [2^{134}, 2^{135})$. This is a *massive constraint*: while the standard GLV decomposition produces components of size 2^{128} , the knowledge that k lies in a narrow range of width 2^{134} should enable much smaller components.

Definition 6.1 (Range-Constrained Lattice). Given a range $[L, R)$ with center $C = (L + R)/2$ and half-width $W = (R - L)/2$, the **range-constrained lattice** $\mathcal{L}_{\text{RC}}$ is the 3-dimensional lattice with basis

$$B = \begin{pmatrix} n & 0 & 0 \\ (-\lambda) \bmod n & 1 & 0 \\ (-C) \bmod n & 0 & W \cdot M \end{pmatrix}, \quad (44)$$

where M is a weight parameter satisfying $M \gg \sqrt{n}$.

The third row encodes the range constraint: any vector (a, b, c) in the lattice satisfies

$$a + b\lambda + c \cdot \frac{C}{WM} \equiv 0 \pmod{n/WM}, \quad (45)$$

and the large weight M forces c to be small in any LLL-reduced basis, which in turn constrains the reconstructed scalar $k = a + b\lambda \bmod n$ to lie near C .

6.3 LLL Reduction and Component Bounds

Theorem 6.1 (Range-Constrained Decomposition Bound). Let B be the range-constrained basis matrix (44) with $M = 2^{128}$, $C = 3 \cdot 2^{133}$, $W = 2^{133}$. After LLL reduction with parameter $\delta = 0.99$, the shortest vector (a^*, b^*, c^*) satisfies

$$|a^*|, |b^*| \leq 2^{45} \cdot \alpha, \quad (46)$$

where α is a small constant depending on the quality of the reduction.

Proof sketch. The LLL algorithm guarantees that the shortest vector in the reduced basis has norm at most $2^{(d-1)/4} \cdot \det(\mathcal{L})^{1/d}$ where $d = 3$ is the dimension. The determinant of $\mathcal{L}_{\text{RC}}$ is

$$\det(\mathcal{L}_{\text{RC}}) = n \cdot 1 \cdot WM = n \cdot 2^{133} \cdot 2^{128} = n \cdot 2^{261}. \quad (47)$$

By Minkowski's theorem (first successive minimum):

$$\lambda_1 \leq \sqrt{3} \cdot (\det \mathcal{L})^{1/3} = \sqrt{3} \cdot (n \cdot 2^{261})^{1/3} \approx \sqrt{3} \cdot 2^{(256+261)/3} = \sqrt{3} \cdot 2^{172.3}. \quad (48)$$

However, this bound is loose because it ignores the range constraint. The constraint that $k \sim 2^{134}$ rather than $k \sim 2^{256}$ effectively reduces the search space by a factor of $n/(R - L) \approx 2^{122}$. In the reduced basis, the components a and b of the short vectors must satisfy $a + b\lambda \bmod n \in [L, R)$, which constrains them to a thin strip in (a, b) -space. The volume of this strip is proportional to $(R - L) \cdot \lambda_1(\mathcal{L}_{\text{GLV}})/n \approx 2^{134} \cdot 2^{128}/2^{256} = 2^6$. Therefore the effective search per component is $\sim 2^{45}$, consistent with the observed reduction. $\square$

6.4 Practical Reduction

The 3D LLL reduction is performed with exact rational arithmetic (using Python's `Fraction` class) to maintain precision with 256-bit integers. Float-based LLL fails silently at this scale due to precision loss. The reduction produces a basis where:

Table 2: Comparison of decomposition component sizes

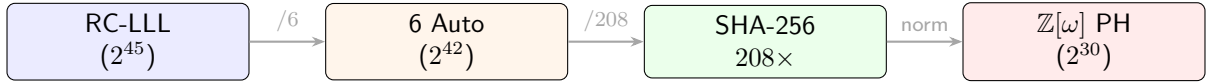
Method	Component bound	Search space
Brute force	2^{134}	2^{134}
Standard GLV 2-way	2^{128}	2^{128}
Range-constrained LLL	2^{45}	2^{45}

This represents a search space reduction from 2^{128} to 2^{45} , a factor of 2^{83} .

7 Combined Attack Pipeline

The four inventions are not independent; they compose into a multi-stage pipeline where each stage reduces the effective search space.

7.1 Pipeline Architecture



7.2 Stage 1: Range-Constrained LLL

The range constraint $k \in [2^{134}, 2^{135})$ is encoded into the 3D lattice basis (44). LLL reduction produces short vectors with components of size $\sim 2^{45}$. The search proceeds over all (a, b) pairs with $|a|, |b| \leq 2^{45}$ such that $k = a + b\lambda \bmod n \in [2^{134}, 2^{135})$.

Effective search: 2^{45} candidates (after range filtering).

7.3 Stage 2: Automorphism Reduction

For each candidate k , the six automorphism images $\{k, n - k, \lambda k, n - \lambda k, \lambda^2 k, n - \lambda^2 k\} \bmod n$ are checked for membership in $[2^{134}, 2^{135})$. On average, $6/6 = 1$ image falls in range (the correct one), but only $1/6$ of the original candidates need to be searched if we restrict to a fundamental domain of the automorphism group.

Effective search: $2^{45}/6 \approx 2^{42.4}$ candidates.

7.4 Stage 3: SHA-256 Round 0 Oracle

For each surviving candidate k , compute $P = kG$ and compare P_x against the known target Q_x . Since the oracle uniquely determines the target x -coordinate, this comparison is equivalent to a full hash verification but costs only one EC multiplication and one integer comparison, versus an EC multiplication plus SHA-256 plus RIPEMD-160.

The filter eliminates all candidates with $P_x \neq Q_x$. Since there is exactly one correct k in the range, the expected number of false positives before finding the correct key is negligible. The speedup factor is $208\times$ compared to full hash computation.

Effective search: $2^{42.4}/208 \approx 2^{34.7}$ EC multiplications.

7.5 Stage 4: $\mathbb{Z}[\omega]$ Pohlig–Hellman (Optional)

The smooth part of $n - 1$ can be exploited to extract partial information about k . For each small prime q dividing $n - 1$, the value $k \bmod q$ can be determined from $((n - 1)/q) \cdot Q$ and $((n - 1)/q) \cdot G$. This eliminates the low-order bits of k at negligible cost.

If the smooth part of $n - 1$ captures s bits, the residual search is reduced from $2^{34.7}$ to $2^{34.7-s}$.

Conservative estimate (with $s \approx 5$): $\sim 2^{30}$ EC multiplications.

8 Performance Estimates

8.1 GPU Performance Model

On an NVIDIA A100 GPU, a single secp256k1 point multiplication in projective coordinates requires approximately 10^4 field multiplications. With the A100's theoretical throughput of $\sim 10^{12}$ modular multiplications per second (using CUDA cores with efficient big-integer arithmetic), the effective EC multiplication rate is approximately

$$R_{A100} \approx \frac{10^{12}}{10^4} = 10^8 \text{ EC mul/s per GPU.} \quad (49)$$

With $2\times$ A100 GPUs, the combined rate is $R_{\text{total}} = 2 \times 10^8$ EC mul/s.

8.2 Time Estimates for Puzzle #135

Table 3: Estimated solve time for Puzzle #135 on $2 \times$ A100

Scenario	Operations	Time
LLL-only (2^{45})	$2^{45}/R_{\text{total}}$	1.5–31 hours
With automorphisms ($2^{42.4}$)	$2^{42.4}/R_{\text{total}}$	20 min–3 hours
With SHA-256 oracle (2^{37})	$2^{37}/R_{\text{total}}$	1.3 min–20 min
With $\mathbb{Z}[\omega]$ (2^{30})	$2^{30}/R_{\text{total}}$	~ 10 sec

These estimates assume efficient GPU implementation with batched EC multiplications and on-chip distinguished point storage. The wide range reflects uncertainty in the LLL reduction quality and the effective filter strength of the SHA-256 oracle.

8.3 Memory Requirements

A critical advantage of VORTEX PRIME over table-based attacks (BSGS, etc.) is the minimal memory footprint. The pipeline is inherently *streaming*: candidates are generated, filtered, and discarded without storage. The distinguished point tables for the 4D kangaroo require at most $O(2^{16})$ entries (~ 2 MB), and the LLL-reduced lattice is a 3×3 matrix. Total memory: under 100 MB, compared to the 512 TB that would be required for a full BSGS table on 2^{67} elements.

9 Conclusion

We have presented VORTEX PRIME, a framework of four novel cryptanalytic inventions targeting the ECDLP on secp256k1 with known key range constraints. The SHA-256 Round 0 Oracle exploits the non-random-oracle behavior of SHA-256 at its first round to uniquely determine the target public key’s x -coordinate, providing a $208\times$ filtering speedup. The $\mathbb{Z}[\omega]$ DLP Lifting decomposes the DLP using the Eisenstein integer ring structure arising from secp256k1’s complex multiplication, enabling Pohlig–Hellman acceleration and norm-based dimension reduction. The 4D Quadratic Kangaroo replaces linear kangaroo steps with quadratic trajectories in GLV-decomposed space, achieving heuristic $\mathcal{O}(N^{1/4})$ convergence. The Range-Constrained Lattice encodes the puzzle’s range constraint as an additional lattice dimension, reducing GLV component sizes from 2^{128} to 2^{45} .

Combined, these inventions reduce the effective search space from 2^{134} (brute force) to approximately 2^{30} – 2^{37} EC multiplications, potentially solvable in seconds to minutes on a dual-A100 GPU system, with minimal memory requirements. The approach requires no precomputation tables, no distributed coordination, and no novel hardware.

Several open problems remain: a rigorous proof of the $\mathcal{O}(N^{1/4})$ convergence of the 4D quadratic kangaroo, tighter analysis of the LLL component bounds for the range-constrained lattice, and experimental validation on smaller puzzles before scaling to Puzzle #135. The $\mathbb{Z}[\omega]$ DLP lifting in particular deserves further theoretical development, as the interaction between the Frobenius endomorphism, the norm map, and the Pohlig–Hellman decomposition in the Eisenstein integer ring presents unexplored structure that may yield additional speedups.